Université Mohammed V de Rabat
Faculté des Sciences de Rabat
Master 1 — Cryptographie et Sécurité de l'Information (CRYPTIS)
Année universitaire 2024–2025

MODULE **Advanced Cryptography** | Prof. Ahmed El-Yahyaoui

Padding Oracle Attack

A Complete Guide

From symmetric encryption fundamentals to plaintext recovery without the key

> *"Mastery requires only two things: time to learn and the willpower to keep answering your questions and deepening your understanding."*
>
> — Oussama Mejdoubi

Submitted by
Oussama Mejdoubi
Master 1 — CRYPTIS
Faculté des Sciences de Rabat

Supervised by
Prof. Ahmed El-Yahyaoui
Advanced Cryptography
Academic Year 2024–2025

A complete, structured guide to the Padding Oracle Attack

Contents

1 Real-World Context: How Modern Encryption Works

Before we talk about the attack, let us understand what we are attacking. Almost every encrypted message you send — emails, passwords, bank transactions — goes through this pipeline.

The journey of an encrypted message

Your browser or app wants to send a secret message to a server. Here's what happens:

1. **User has message**
2. **Message gets encrypted** (using a symmetric block cipher)
3. **Encrypted data sent** over network
4. **Server receives** encrypted data
5. **Server decrypts it** (using the same cipher and key)
6. **Server reads** the message
7. **Server sends response** (encrypted)

The key question: If someone intercepts the encrypted message, can they read it? **No.** Without the encryption key, it is gibberish. Unless... there is a flaw in how the decryption works.

What we will discover: A tiny flaw — the server accidentally leaking information about whether a decrypted message's padding is valid — is enough to decrypt any message, even *without the key*.

Why this matters

This is not theoretical. Real systems have been vulnerable:

- **ASP.NET (2010):** Microsoft emergency patch after attackers exploited this flaw against web applications.
- **TLS/SSL (2011–2013):** SSL libraries leaked timing information that exposed the same vulnerability.

- **Today:** Any system using CBC mode encryption (still common) without proper safeguards is at risk.

2 Symmetric Encryption Using Block Ciphers

To understand the attack, you first need to understand the system being attacked. Symmetric encryption is the backbone of nearly all secure communication on the internet — from HTTPS to encrypted file storage. Let us break it down from first principles.

What is symmetric encryption?

Symmetric encryption means **the same key** is used to both encrypt and decrypt. Think of it like a physical lockbox where the sender and receiver share an identical key.

Property	Meaning
Same key both ways	Whoever can encrypt can also decrypt
Fast & efficient	Well-suited for large volumes of data
Key must stay secret	If the key leaks, all messages are exposed

What is a block cipher?

A **block cipher** is a specific kind of symmetric encryption algorithm. Instead of encrypting a message one bit at a time (stream cipher), it slices the message into fixed-size **blocks** and encrypts each block as a unit using the secret key.

The core operation: given a block of plaintext P and a secret key K, the cipher produces a block of ciphertext C:

$$C = E_K(P) \qquad \text{and} \qquad P = D_K(C)$$

Both E (encrypt) and D (decrypt) are deterministic, invertible functions. Without K they are computationally infeasible to reverse.

A block cipher as a black box

It is easiest to think of a block cipher as a locked black box:

```
ENCRYPT:
 INPUT: plaintext block (e.g.  16 bytes) + SECRET KEY
  [CIPHER BOX]
 OUTPUT: ciphertext block (16 bytes of gibberish)

DECRYPT:
 INPUT: ciphertext block (16 bytes) + SECRET KEY
  [CIPHER BOX]
 OUTPUT: original plaintext block (recovered exactly)
```

Without the secret key, even knowing the exact algorithm gives you nothing — the output looks like random noise.

Block size and key size

Every block cipher has two fundamental parameters:

- **Block size:** the fixed number of bytes processed in one operation (commonly 16 bytes / 128 bits). A message must be a whole number of blocks — this is why **padding** is needed.
- **Key size:** the length of the secret key (e.g. 128, 192, or 256 bits). Larger keys mean exponentially more brute-force work for an attacker.

Well-known block ciphers

Many block ciphers have been designed and standardised over the decades:

Cipher	Block size	Key size	Status
DES	64 bits	56 bits	**Broken** (too small)
3DES	64 bits	112 / 168 bits	**Legacy** (slow, deprecated)
AES	128 bits	128 / 192 / 256 bits	**Current standard**
Camellia	128 bits	128 / 192 / 256 bits	**Widely deployed**

Why this matters for our attack: the padding oracle attack does **not** break the block cipher itself. AES, for example, has no known weakness. The attack instead exploits **how the cipher is used** — specifically the padding scheme applied before encryption and the error messages revealed during decryption.

The fundamental limitation: block ciphers need a mode

A bare block cipher only encrypts *one fixed-size block*. Real messages are longer and vary in size. Two problems arise immediately:

1. **Size mismatch:** if the message isn't a perfect multiple of the block size, you need to pad it to fit — this is where PKCS#7 padding comes in (Section 4).

2. **Pattern leakage:** encrypting two identical blocks with the same key always produces the same ciphertext block. An attacker watching ciphertext can spot repeated patterns in the message. To fix this, you need a **mode of operation** — the most common being CBC, covered in the next section.

> **Key insight:** a block cipher is just the primitive. The **mode of operation** is the recipe for applying it safely to real-world messages. Weaknesses almost always live in the mode, not the cipher — and the padding oracle attack is a perfect example.

3 What is CBC Mode? (Chaining Blocks Together)

CBC = Cipher Block Chaining. It's a method to chain multiple blocks together so patterns don't repeat. It can be applied on top of *any* block cipher.

Why do we need modes?

If you encrypted two identical blocks with a bare block cipher, you'd get identical ciphertext blocks. That's bad — an attacker can see patterns in the ciphertext that reveal patterns in the plaintext. CBC solves this by **mixing each plaintext block with the previous ciphertext block** before encrypting, so identical inputs produce different outputs every time.

How CBC encryption works (simplified)

Imagine encrypting a 16-byte message `"helloworld12345"`:

```
STEP 1:   Generate a random 16-byte IV (Initialization Vector)
 IV = "randomrandom1234" (never reused, sent openly)
STEP 2:   XOR the first plaintext block with the IV
 plaintext XOR IV = "mixed_stuff"
STEP 3:   Encrypt the XOR result with the block cipher
 BlockCipher_encrypt("mixed_stuff", key) = "ciphertext_block"
```

```
OUTPUT:  IV + ciphertext_block (both transmitted to receiver)
```

For multi-block messages, each ciphertext block feeds into the next XOR, forming the "chain" that gives CBC its name.

Key point: The IV is random and unique per message, so the same plaintext encrypted with the same key will produce a **completely different ciphertext** every time. The IV does not need to be secret — only the key does.

For decryption (the reverse):

The receiver gets `IV + ciphertext`, then for each block:

1. **Decrypt** the ciphertext block with the secret key → get the intermediate state $D(C)$
2. **XOR** the intermediate state with the previous ciphertext block (or IV for the first block) → get the original plaintext block

Critical detail: this XOR step in decryption is exactly where the padding oracle attack operates. The attacker can **control the IV** (or the preceding ciphertext block), which directly influences what plaintext the server sees after decryption. This control is the foundation of the entire attack.

4 Padding: Making Messages Fit

Block ciphers work with **fixed-size blocks**. What if your message doesn't fill a block exactly? You need **padding** — extra bytes appended to the message to make its length a perfect multiple of the block size. The clever part of PKCS#7 padding: each padding byte encodes how many padding bytes were added, so the receiver can strip them cleanly.

PKCS#7 Padding Rule

Append N bytes, where each byte has value N (the number of padding bytes needed). This way, the server can read the last byte and immediately know how many padding bytes to remove.

Case 1: Message needs 1 byte of padding (`0x01`)

```
Message:  "abcdefghijklmno" (15 bytes)
Need:  16 - 15 = 1 byte padding
Padding:  1 byte with value 0x01
```

```
FINAL: "abcdefghijklmno" + [0x01] = 16 bytes
```

Case 2: Message needs 11 bytes of padding (0x0B)

```
Message:  "hello" (5 bytes)
Need:  16 - 5 = 11 bytes padding
Padding:  11 bytes, each with value 0x0B
FINAL: "hello" + [0x0B  11] = 16 bytes
```

Case 3: Message is exactly 16 bytes (block is FULL)

The ambiguity problem: Without a full padding block, the server reads the last byte and cannot tell if it is data or padding. **CONFUSION!**

Solution: Always add a whole new padding block!
Message `"complete16bytes!"` (16 bytes) + Padding `[0x10  16]` = 32 bytes total.

Now on decryption, server reads last byte = `0x10` (16) and knows: "remove 16 bytes."

Summary: All major padding cases

Message length	Padding	Total
1 byte	`[0x0F  15]`	16 bytes
8 bytes	`[0x08  8]`	16 bytes
15 bytes	`[0x01  1]`	16 bytes
16 bytes	`[0x10  16]`	32 bytes

Critical: If the server receives a message with **invalid padding**, it throws an error. This tiny error message is the **start of our attack**.

Realistic Scenario: How Padding and Unpadding Actually Work

Key question answered here: Is padding removed after each block is decrypted, or only at the very end? The answer is **only at the very end** — and this scenario shows exactly why.

Setup: block size = 16 bytes, CBC mode, PKCS#7 padding applied *once* to the full message stream.

Input messages:

- Message A = `"123456"` (6 bytes)
- Message B = `"1234567890123456"` (16 bytes)

Step 1 — Concatenate into one continuous byte stream.

CBC does not treat messages separately. The two messages are joined end-to-end before any block splitting occurs:

$$\text{Plaintext stream} = A \parallel B = \texttt{"12345612345678901234567890123456"} \quad (22 \text{ bytes})$$

Step 2 — Split into 16-byte blocks (before padding).

```
Block 1 (complete):   "1234561234567890" (16 bytes)
Block 2 (partial):   "123456" (6 bytes - needs padding)
```

Step 3 — Apply PKCS#7 padding to complete Block 2.

$$\text{Missing bytes} = 16 - 6 = 10 \quad \Rightarrow \quad \text{padding byte} = \texttt{0x0A}$$

```
P1 = 31 32 33 34 35 36 31 32 33 34 35 36 37 38 39 30
P2 = 31 32 33 34 35 36  0A 0A 0A 0A 0A 0A 0A 0A 0A 0A
 [6 data bytes] [10 padding bytes, value 0x0A each]
```

Step 4 — CBC Encryption.

$$C_1 = E(P_1 \oplus IV) \qquad C_2 = E(P_2 \oplus C_1)$$
$$\text{Transmitted:} \quad IV \parallel C_1 \parallel C_2$$

Step 5 — CBC Decryption (receiver side).

Block	Intermediate state	XOR to recover plaintext
Block 1	$X_1 = D(C_1)$	$P_1 = X_1 \oplus IV$
Block 2	$X_2 = D(C_2)$	$P_2 = X_2 \oplus C_1$

After decryption, the receiver holds both plaintext blocks in raw form:

$$P_1 = \texttt{"1234561234567890"} \qquad P_2 = \texttt{"123456"} + \texttt{0x0A} \times 10$$

Step 6 — Reconstruct the full plaintext stream by concatenating.

$$P_1 \parallel P_2 = \texttt{"12345612345678901234567890123456"} + \texttt{0x0A} \times 10$$

Step 7 — PKCS#7 Unpadding (applied once, on the last block only).

Rule: inspect only the last byte of the last block.

Last byte of P_2 = `0x0A` = 10 in decimal.
Verify: are the last 10 bytes all equal to `0x0A`? **Yes.**

Remove 10 bytes from the end of P_2.

Never touch P_1 — it is not the last block.

Final recovered plaintext:

`"12345612345678901234 56"` (22 bytes, exactly the original input)

Three rules that must never be forgotten:

1. **CBC treats all data as one continuous byte stream.** Message boundaries do not exist inside CBC encryption — the cipher sees a single sequence of blocks.
2. **PKCS#7 padding is applied once, at the very end of the full stream.** It fills only the last block, never any intermediate block.
3. **Unpadding happens once, on the last block, after all blocks are decrypted.** Only the last block is inspected; all others are left untouched.

5 Encryption in Action: From User to Server

Let's walk through a real example: a user sending a password to a server.

Example: Encrypting a password

User's browser wants to send password `"MyPass2024"` to server. Here's what happens:

```
1. Create message with padding:
   "MyPass2024" is 10 bytes. Needs 6 bytes padding (0x06 each).
   Padded: "MyPass2024" + [06 06 06 06 06 06] = 16 bytes
2. Generate random IV:
  Browser creates 16 random bytes: "aB#$xY%&@! zQ9mK"
3. Mix with IV:
  padded_msg XOR IV = "mixed_result"
4. Encrypt:
   BlockCipher_encrypt("mixed_result", SECRET_KEY) = "k3x9p2m1d5w8..."
5. Send:
```

```
Browser sends [IV + encrypted_data] over the network.
```

Key insight: The IV doesn't need to be secret — it's random. Only the **SECRET_KEY** is confidential.

6 Decryption: The Server's Side

The server receives `[IV + encrypted_data]` and decrypts. This is where the vulnerability hides.

Complete decryption flow

Step 1. Receive `[IV + ciphertext]`

Step 2. Decrypt ciphertext with secret key → get *intermediate state* $D(C)$

Step 3. XOR intermediate state with IV → get plaintext $P = D(C) \oplus IV$

Step 4. Check padding: is the last byte (or bytes) a valid padding value?

Step 5. Return ✓ VALID or INVALID

The vulnerability: The server's response — *"valid"* or *"invalid"* padding — **leaks information** about the decrypted bytes. An attacker uses this to reverse-engineer the plaintext.

7 The Information Leak: What Goes Wrong

The server is supposed to be secure. But by accident, it tells the attacker: *"that padding is invalid"* or *"that padding is valid"*. This tiny piece of information is **dangerous**.

Why does the server leak this?

The server has to check padding. There are only two outcomes:

Padding is valid	or	Padding is invalid

The problem: An attacker can send **modified ciphertext** and observe which outcome happens. This gives information. The attacker calls this the **padding oracle** — an oracle because it answers yes/no questions about padding validity.

How the leak happens in practice

Different systems leak information in different ways:

- **Error messages:** Server returns `400 Bad Request` vs `500 Internal Error`
- **Timing:** Valid padding decryption takes slightly longer
- **Response size:** Different response lengths for valid/invalid padding

8 The Padding Oracle: Asking Yes/No Questions

An attacker can send modified ciphertext to the server and learn: *"Is this padding valid?"* By asking this question 256 times, they can recover encrypted bytes.

Simple example: Finding one encrypted byte

The attacker has an encrypted message. They want to know: *what is the last encrypted byte?*

```
Step 1:   Modify last byte to 0x00, send.  Server:    Invalid padding
Step 2:   Modify last byte to 0x01, send.  Server:    Invalid padding
Step 3:   Try 0x02 through 0xFF. Server keeps saying:    Invalid
Step 4:   Try 0xD8.  Server:  ✓ Valid padding!
```

Success! The attacker found that when last byte = `0xD8`, padding is valid.

Why this works: The attacker modified the ciphertext but NOT the message inside. They triggered a specific condition in the decrypted data: **valid padding**. This condition happens for exactly one byte value (out of 256). Finding it took at most 256 tries.

What the attacker learned

The attacker now knows: when the last ciphertext byte = `0xD8`, the decrypted message ends with valid padding of type `"01"` (one byte of value `0x01`). Using algebra, they can calculate the original encrypted value.

The magic: By repeating this 16 times (once per byte in a block), the attacker recovers the **entire decrypted block** — without the encryption key!

9 Technical Deep Dive: The Algorithm Step-by-Step

Now let's get rigorous. The padding oracle attack is built on one principle: **build assumptions and verify them**.

Core Principle: Assume & Verify

1. **Build assumption:** "If I set $IV[\text{last}] = \texttt{0xD8}$, the plaintext will end with `0x01`"
2. **Test it:** Send modified ciphertext to the oracle
3. **Verify:** Oracle returns ✓ **VALID** or **INVALID**
4. **Solve:** Use response + your chosen IV to extract the unknown encrypted byte

The Attack Equation (Everything Hinges On This)

CBC decryption, for every byte position $[i]$:

$$P_1[i] \;=\; D(C_1)[i] \;\oplus\; IV[i]$$

- **What you CAN control:** $IV[i]$ — you can set this to any of 256 values
- **What you CANNOT know directly:** $D(C_1)[i]$, the encrypted intermediate state (unknown but **FIXED**)
- **The magic:** by controlling IV and observing the oracle response, you can deduce $D(C_1)[i]$ using pure algebra

Step-by-Step: Recovering Byte 7 (The Last Byte)

Block size = 8 bytes. We target the last byte first.
Original plaintext: `"hello"` followed by `0x03 0x03 0x03` (5 data bytes + 3 padding bytes).

Brute-force the IV: Modify only $IV'[7]$; keep $IV'[0\text{–}6]$ unchanged.
Try all 256 values: $IV'[7] = \texttt{0x00}, \texttt{0x01}, \ldots, \texttt{0xFF}$

After at most 256 attempts, suppose $IV'[7] = \texttt{0xD8}$ returns ✓ **VALID**.

Ambiguity Problem: What does "valid" actually mean?
Case A: you created `0x01` padding — the intended target.
Case B: you accidentally preserved the original `0x03 0x03 0x03` padding.
Both return ✓ VALID — you cannot tell them apart yet.

Verification test (the key trick): Change another byte — set $IV'[6] =$ `0x99` — and resend.

- Still ✓ **VALID** → padding was `0x01` (only the last byte matters for single-byte padding)
- Now **INVALID** → padding was `0x03 0x03 0x03` (a false positive — discard)

Solve the Algebra: You've confirmed $D(C_1)[7] \oplus$ `0xD8` $=$ `0x01`, so:

$$\boxed{D(C_1)[7] \;=\; \texttt{0x01} \oplus \texttt{0xD8} \;=\; \texttt{0xD9}}$$

Why XOR Reversibility is the Key

The entire attack depends on this mathematical property:

XOR is self-reversing: if $a \oplus b = c$, then $a = c \oplus b$.

Example: `0xD9` $\oplus$ `0xD8` $=$ `0x01`, therefore `0xD9` $=$ `0x01` $\oplus$ `0xD8`.

This is how we extract the intermediate byte **without ever knowing the key**.

Repeat for All Remaining Bytes

Once byte 7 is done, target byte 6, then 5, and so on:

Byte 6 (target `0x02 0x02` padding):
Set $IV'[7] = D(C_1)[7] \oplus$ `0x02` (forces last byte to `0x02`)
Brute-force $IV'[6]$ until ✓ **VALID**
Solve: $D(C_1)[6] =$ found $\oplus$ `0x02`

Byte 5 (target `0x03 0x03 0x03` padding):
Set $IV'[7]$ and $IV'[6]$ to their pinned values
Brute-force $IV'[5]$ until ✓ **VALID**
Solve: $D(C_1)[5] =$ found $\oplus$ `0x03`

Continue leftward... At most 256 tries per byte → 2048 requests per block.

The False Positive Problem (The Subtle Part)

When the original plaintext already has valid padding, the oracle can return ✓ **VALID** for the *wrong* reason. There are two distinct sources of false positives, and both must be handled.

Two ways a false positive can arise:

1. **The XOR gives accidental valid padding.** When you XOR your crafted IV with the fixed keystream $D(C_1)$, the result might happen to form a valid padding sequence even though you did not intend it. For example, instead of producing `0x01` at the last byte, the XOR might accidentally produce `0x03 0x03 0x03` — which is also valid PKCS#7 padding.

2. **The original message already ends with valid padding.** If the actual plaintext of the ciphertext block you are targeting already ends with valid padding bytes (for example, the last block C_n decrypts to a message ending in `0x06 0x06 0x06 0x06 0x06 0x06`), then even without any modification the oracle returns ✓ **VALID**. Setting $IV'[\text{last}] =$ `0x00` XOR'd into the keystream may simply reproduce that original valid padding — giving a second valid hit that is not the `0x01` you are looking for.

Overcoming False Positives: A Reliable Strategy

The solution is to combine two complementary techniques into one robust assumption-and-verify loop.

Technique 1 — Exclude the original last ciphertext byte

Why this matters: you are always attacking the *last* ciphertext block, which by definition carries real PKCS#7 padding. The original $IV[-1]$ (the last byte of the preceding block or the IV itself) was specifically chosen during encryption to produce that valid padding. If you ever land on that original byte value while brute-forcing, the oracle says ✓ **VALID** but for the *wrong* reason — the original padding survived your modification unchanged.

Fix: simply skip the value $IV_{\text{original}}[-1]$ during your sweep of $IV'[-1]$. Since you are given the IV (it is sent in plaintext), you always know what to exclude. This costs you nothing — at most 255 meaningful candidates remain.

Technique 2 — Verify by perturbing a nearby byte

Once a candidate value for $IV'[-1]$ triggers ✓ **VALID**, you cannot yet trust it. Apply this verification test before recording the result:

Step 1. Note the candidate: $IV'[-1] = v$ returned ✓ **VALID**.

Step 2. Change the second-to-last byte: set $IV'[-2]$ to any value other than its current one, keeping all other bytes the same.

Step 3. Resend to the oracle.

Interpret the result:

- ✓ **VALID** again → padding is `0x01`. Only the last byte matters for single-byte padding, so changing the second-to-last byte has no effect. **Real hit — record it.**
- **INVALID** → padding was something longer (e.g. `0x03 0x03 0x03`), and your change broke one of its required bytes. **False positive — discard and continue sweeping.**

Recognising which padding value was actually present

The verification test also tells you exactly what the original padding was, which is useful context:

- If the false positive survives until you perturb byte -2 but breaks at byte -3, the original padding was `0x02 0x02`.
- If it breaks only when you perturb byte $-k$, the original padding was k bytes of value k.
- You can use this to skip ahead efficiently: once you know the original padding length k, the last $k-1$ bytes are already known (they all equal k), so you only need to brute-force byte $-(k+1)$ next.

Combining both techniques gives you a clean, reliable loop:

1. Sweep $IV'[-1]$ through all 256 values, **skipping** $IV_{\text{original}}[-1]$.
2. For each ✓ **VALID** hit, apply the perturbation test on $IV'[-2]$.
3. Accept only hits that remain ✓ **VALID** after the perturbation.
4. Solve: $D(C_1)[-1] = \texttt{0x01} \oplus IV'[-1]$.
5. Proceed left, using the already-recovered bytes to pin the padding suffix.

The two techniques together eliminate both classes of false positives.

Note: To verify this, a Python proof-of-concept is available at:
https://github.com/sam99235/padding-oracle-attack

10 Putting It Together: The Complete Attack

10.1 Recovering the Remaining Bytes: A Rigorous Walk-Through

The Setup: What the Attacker Sends

The attacker captures one ciphertext block and submits it paired with a crafted **IV of all zeros**. The table below shows every layer of the server's decryption routine. Two rows are *unknown* to the attacker at the start; one row is fully **controlled**; and the last row is what the server computes internally.

Row	Bytes (positions 0 – 7)							
Ciphertext C_1	`7c`	`d3`	`dc`	`3e`	`98`	`cb`	`52`	`81`
$D(C_1)$ *(unknown)*	`??`	`??`	`??`	`??`	`??`	`??`	`??`	`??`
IV (attacker controls)	`00`	`00`	`00`	`00`	`00`	`00`	`00`	`00`
$P_1 = D(C_1) \oplus IV$ *(unknown)*	`??`	`??`	`??`	`??`	`??`	`??`	`??`	`??`

Key observation: Because IV_{original} = `0x00` for every byte, the equation $P_1[i] = D(C_1)[i] \oplus$ `0x00` simplifies to $P_1[i] = D(C_1)[i]$. Whatever we recover as the **intermediate state** is *directly* the plaintext in this demo setup.

Step 1 — Recovering Byte 7 (Last Byte)

We sweep $IV'[7]$ through all 256 values (`0x00` → `0xFF`) while keeping bytes 0–6 at `0x00`. For each attempt we send the pair (IV', C_1) to the padding oracle and wait for its single-bit answer.

After at most 256 attempts, suppose the oracle returns ✓ **VALID** when $IV'[7] =$ `0xD8`. This tells us:

$$P_1[7] = \texttt{0x01} \implies D(C_1)[7] \oplus \underbrace{\texttt{0xD8}}_{\text{our } IV'[7]} = \texttt{0x01}$$

Applying XOR's self-inverse property ($x \oplus a = b \Rightarrow x = b \oplus a$):

$$\boxed{D(C_1)[7] = \texttt{0x01} \oplus \texttt{0xD8} = \texttt{0xD9}}$$

What we just learned: The **keystream byte** $D(C_1)[7] = \texttt{0xD9}$ is now **known**. It is a fixed property of the ciphertext block — it will never change, regardless of what IV we craft next. We will use it in every subsequent round.

The state of our knowledge after Round 1:

Row	Bytes (positions 0 – 7)							
$D(C_1)$	??	??	??	??	??	??	??	d9

The Chain Reaction: From One Byte to All Bytes

Here is the central insight that makes the whole attack possible:

Because we now **know** $D(C_1)[7]$, we can **force** position 7 to produce *any* plaintext byte we choose, simply by setting:

$$IV'[7] \;=\; \text{desired value} \;\oplus\; D(C_1)[7]$$

In particular, to target **padding value** 0x02, we need $P_1[7] = \texttt{0x02}$, so:

$$IV'[7] \;=\; \texttt{0x02} \oplus \texttt{0xD9} \;=\; \texttt{0xDB}$$

With byte 7 *pinned* to 0x02, the oracle will only return ✓ **VALID** when byte 6 *also* equals 0x02 — giving us a clean, unambiguous signal to find $D(C_1)[6]$.

Step 2 — Recovering Byte 6 (Target Padding 0x02)

Goal: find $D(C_1)[6]$ by engineering the suffix $P_1[6]\,P_1[7] = \texttt{0x02}\,\texttt{0x02}$.

Fix byte 7. We already know $D(C_1)[7] = \texttt{0xD9}$, so:

$$IV'[7] \;=\; \texttt{0x02} \oplus D(C_1)[7] \;=\; \texttt{0x02} \oplus \texttt{0xD9} \;=\; \texttt{0xDB} \quad\Rightarrow\quad P_1[7] = \texttt{0x02}\ \checkmark$$

Sweep byte 6. With $IV'[7]$ locked, sweep $IV'[6]$ through 0x00 → 0xFF. Suppose the oracle returns ✓ **VALID** when $IV'[6] = \texttt{0x22}$.

Solve:

$$D(C_1)[6] \oplus \texttt{0x22} = \texttt{0x02} \quad\Longrightarrow\quad \boxed{D(C_1)[6] \;=\; \texttt{0x02} \oplus \texttt{0x22} \;=\; \texttt{0x20}}$$

Updated state of knowledge:

Row	Bytes (positions 0 – 7)							
$D(C_1)$	??	??	??	??	??	??	20	d9

Step 3 — Recovering Byte 5 (Target Padding 0x03)

Goal: engineer the suffix $P_1[5]\,P_1[6]\,P_1[7] = \texttt{0x03}\,\texttt{0x03}\,\texttt{0x03}$.

Fix bytes 6 and 7 using the two keystream bytes we now own:

$$IV'[7] = \texttt{0x03} \oplus D(C_1)[7] = \texttt{0x03} \oplus \texttt{0xD9} = \texttt{0xDA} \quad \Rightarrow \quad P_1[7] = \texttt{0x03}\ \checkmark$$
$$IV'[6] = \texttt{0x03} \oplus D(C_1)[6] = \texttt{0x03} \oplus \texttt{0x20} = \texttt{0x23} \quad \Rightarrow \quad P_1[6] = \texttt{0x03}\ \checkmark$$

Sweep byte 5. Suppose $IV'[5] = \texttt{0x79}$ returns ✓ **VALID**.

Solve:

$$\boxed{D(C_1)[5] \;=\; \texttt{0x03} \oplus \texttt{0x79} \;=\; \texttt{0x7A}}$$

Updated state of knowledge:

Row	Bytes (positions 0 – 7)							
$D(C_1)$	??	??	??	??	??	7a	20	d9

Step 4 — Recovering Byte 4 (Target Padding 0x04)

Goal: engineer the suffix $P_1[4] \ldots P_1[7] = \texttt{0x04}\,\texttt{0x04}\,\texttt{0x04}\,\texttt{0x04}$.

Fix bytes 5, 6, 7:

$$IV'[7] = \texttt{0x04} \oplus \texttt{0xD9} = \texttt{0xDD}$$
$$IV'[6] = \texttt{0x04} \oplus \texttt{0x20} = \texttt{0x24}$$
$$IV'[5] = \texttt{0x04} \oplus \texttt{0x7A} = \texttt{0x7E}$$

Sweep byte 4. Suppose $IV'[4] = \texttt{0x9E}$ returns ✓ **VALID**.

Solve:

$$\boxed{D(C_1)[4] \;=\; \texttt{0x04} \oplus \texttt{0x9E} \;=\; \texttt{0x9A}}$$

The General Pattern

By now the pattern is clear. At every round r (targeting byte $k = 7 - r$, padding value $v = r + 1$):

1. **Pin all already-recovered bytes** to value v:

   $$IV'[j] \;=\; v \;\oplus\; D(C_1)[j] \qquad \forall\, j > k$$

   This is possible *only because* we know $D(C_1)[j]$ from previous rounds.

2. **Sweep** $IV'[k]$ through `0x00` → `0xFF`.

3. When the oracle says ✓ **VALID**, **solve**:

   $$D(C_1)[k] \;=\; v \;\oplus\; IV'[k]$$

4. **Recover plaintext** using the original IV:

   $$P_1[k] \;=\; D(C_1)[k] \;\oplus\; IV_{\text{original}}[k]$$

Complete Recovery Table

Running all 8 rounds to completion, every byte of $D(C_1)$ falls out. Since $IV_{\text{original}} = \texttt{0x00}$ throughout, the plaintext equals the intermediate state directly.

Round	Target	Padding v	Found $IV'[k]$	$D(C_1)[k]$	Plaintext $P_1[k]$
1	byte 7	0x01	0xD8	0xD9	0xD9
2	byte 6	0x02	0x22	0x20	0x20 (SPACE)
3	byte 5	0x03	0x79	0x7A	0x7A (z)
4	byte 4	0x04	0x9E	0x9A	0x9A
5	byte 3	0x05	0xF3	0xF6	0xF6
6	byte 2	0x06	0xE6	0xE0	0xE0
7	byte 1	0x07	0x9A	0x9D	0x9D
8	byte 0	0x08	0x47	0x4F	0x4F (O)

And here is the **fully recovered intermediate state** $D(C_1)$, which we started knowing nothing about:

Row	Bytes (positions 0 – 7)							
Ciphertext C_1	7c	d3	dc	3e	98	cb	52	81
$D(C_1)$ **(fully recovered!)**	4f	9d	e0	f6	9a	7a	20	d9
IV (original, all zeros)	00	00	00	00	00	00	00	00
$P_1 = D(C_1) \oplus IV$ **(plaintext!)**	4f	9d	e0	f6	9a	7a	20	d9

Remarkable fact: The attacker never needed the encryption key. They never broke the block cipher. All they needed was the ciphertext, the ability to resend modified versions, and the server's one-bit ✓ **VALID**/ **INVALID** answer about padding. That single bit, asked up to $256 \times 8 = 2048$ times, was enough to expose every byte of the block.

11 Decrypting Multiple Ciphertext Blocks

Important recall: PKCS#7 padding is only ever present at the **very end** of the full message — in the last ciphertext block. All preceding blocks carry pure plaintext data, no padding bytes at all. This is what determines the order in which you attack the blocks.

So far the complete attack has shown how to decrypt a single ciphertext block C_1 from the simple case $IV \| C_1$. In practice, an intercepted message looks like:

$$IV \parallel C_1 \parallel C_2 \parallel C_3 \parallel \cdots \parallel C_n$$

How do you decrypt all of them?

The key observation: every block pair is an independent oracle query

In CBC decryption, each plaintext block is recovered as:

$$P_i \;=\; D(C_i) \;\oplus\; C_{i-1} \qquad (P_1 = D(C_1) \oplus IV)$$

The preceding block C_{i-1} plays exactly the same role as the IV did in the single-block case. The attacker can **craft and substitute** C_{i-1} just as freely as the IV, because the server only needs a pair (C_{i-1}, C_i) to perform a decryption and report a padding result. The rest of the ciphertext is irrelevant to the oracle query.

This means the attack generalises immediately: **to decrypt block C_i, treat C_{i-1} as your controllable IV** and run the exact same 256-per-byte brute-force loop.

Two practical strategies

Strategy A — Attack from the last block backwards (recommended)

Why start at the end?
The last block C_n is the only one that carries padding. Because PKCS#7 padding is always at the tail of the message, the oracle's padding check is always performed on the decryption of C_n. Starting there is natural and requires no reordering.

Order of attack:

1. Decrypt C_n using C_{n-1} as the controllable predecessor.
2. Strip the recovered padding from P_n to find the real last bytes.
3. Decrypt C_{n-1} using C_{n-2} as the controllable predecessor.

4. Continue leftward until C_1 is recovered using the real IV.

Advantage: you never need to touch the ciphertext order. You always send the original pair (C_{i-1}, C_i) to the server, modifying only C_{i-1} byte by byte. Once C_i is fully decrypted, discard it and move left.

Strategy B — Reorder blocks to isolate any target

How it works:
To decrypt an arbitrary block C_k, simply send the pair (C_{k-1}, C_k) to the server as if it were the entire ciphertext. The server will decrypt C_k and check the padding of the result. You brute-force C_{k-1} byte by byte as your controllable predecessor.

Advantage: you can target any block in any order — useful if you only care about a specific part of the plaintext (e.g. a session token at a known offset).
Trade-off: the decrypted P_k you recover will have its last bytes interpreted as padding, which may introduce garbage if C_k is not the original last block. You must account for this when reassembling the plaintext.

Summary for n blocks:
Total oracle queries $\leq 256 \times B \times n$, where B is the block size in bytes.
For 16-byte blocks: at most $256 \times 16 \times n = 4096n$ requests to fully decrypt the entire message — still no key required.

Epilogue: Why This is Called a "Side-Channel" Attack

The underlying block cipher stays unbroken. The attacker exploits a side-effect — the server leaking padding validity through error messages, timing, or response size. It is like cracking a safe not by forcing the lock, but by listening to the clicks as the dial turns. The weakness lies entirely in **how the cipher is used**, not in the cipher itself.

Real-world impact: In 2010, Microsoft shipped an emergency patch (MS10-070) because ASP.NET was vulnerable to exactly this attack, putting thousands of web apps at risk.

How to Defend Against It

- **Use AEAD encryption:** modes like AES-GCM or ChaCha20-Poly1305 authenticate the ciphertext. Modify one bit and decryption fails outright — no padding step to leak from.

- **Or:** make valid and invalid padding indistinguishable — identical responses and constant-time comparison, so there's no signal to measure.